

OOPS Interview Questions

140 Comprehensive Questions with Practical Code Answers & Explanations

STAGE 1: OOP Fundamentals

STAGE 2: Encapsulation

STAGE 3: Inheritance

STAGE 4: Polymorphism

STAGE 5: Abstraction

STAGE 6: Object Relationships

STAGE 7: Important OOP Keywords

STAGE 8: Comparison-Based Topics

STAGE 1: OOP Fundamentals

A. Class & Object Basics

1. What is a class in Java?

Answer: A class is a blueprint or template for creating objects. It defines state (attributes) and behavior (methods) of objects. Think of it as a cookie cutter, while objects are the cookies.

```
class Car {
    String brand;
    int speed;

    void drive() {
        System.out.println(brand + " is driving at " + speed + " km/h");
    }
}
```

2. What is an object in Java?

Answer: An object is a real-world instance of a class. It has its own state stored in memory and can use the class's methods to perform behavior.

```
Car myCar = new Car();
myCar.brand = "BMW";
myCar.speed = 100;
myCar.drive(); // BMW is driving at 100 km/h
```

3. How do you create an object in Java?

Answer: Objects are created using the new keyword, which allocates memory & calls the constructor.

```
Car car1 = new Car(); // Object created
Car car2 = new Car(); // Another object
```

4. What is the difference between a class and an object?

Class	Object
Blueprint or template	Instance of a class
Defines state and behavior	Holds actual values for state
Exists in code	Exists in memory (heap)
Can't perform action by itself	Can perform actions using methods

Example: Car is the class, myCar is the object.

5. What is the purpose of the new keyword in Java?

Answer:

- Allocates memory on the heap for a new object.
- Calls the constructor to initialize the object.

```
Car car = new Car(); // 'new' allocates memory and calls constructor
```

6. What happens in memory when an object is created?

Answer:

1. Memory allocated on heap for instance variables.
2. The constructor initializes variables.
3. The reference variable points to the memory location in the stack.

Stack: car -> Heap: Car object {brand: null, speed: 0}

7. Can you have a class without any attributes or methods?

Answer: Yes, Java allows empty classes. Sometimes used as marker classes.

```
class Marker {}
```

8. What are instance variables and instance methods?

Instance Variables:

- **Definition:** Variables that belong to an instance (object) of a class.
- **Scope:** Each object of the class has its own copy of instance variables.
- **Declaration:** Declared inside the class but outside any method, usually without `static`.
- **Access:** Accessed through the object reference.

```
class Car {
    String color; // instance variable
    int speed;    // instance variable
}

Car car1 = new Car();
Car car2 = new Car();
car1.color = "Red";
car2.color = "Blue";
// car1.color is "Red" and car2.color is "Blue" -> each object has its own copy.
```

Instance Methods:

- **Definition:** Methods that belong to an instance (object) of a class.
- **Access:** Can access instance variables and other instance methods directly.
- **Declaration:** Normal methods without the `static` keyword.

```
class Car {
    String color;
    void displayColor() { // instance method
        System.out.println("Car color is: " + color);
    }
}
```

9. What is a static variable and how is it different from an instance variable?

Answer: A static variable (class variable) belongs to the class rather than to any specific object. It is declared using the `static` keyword. All objects share the same copy.

```
class Car {
    static int numberOfCars; // shared by all Car objects
}
```

Feature	Instance Variable	Static Variable
Belongs to	Object (each object has its own copy)	Class (shared by all objects)
Memory Allocation	Allocated when object is created	Allocated only once when class is loaded
Access	Through object reference	Through class name (or object)
Purpose	Store object-specific data	Store class-level/shared data

10. Can static methods access instance variables? Why or why not?

Answer: No, static methods belong to the class, not any object. They cannot access instance variables directly because instance variables belong to objects.

```
class Demo {
    int x = 10;
    static void printX() {
        // System.out.println(x); // Error: Cannot make a static reference to non-static field x
    }
}
```

B. Constructors & Initialization

11. What is a constructor in Java?

Answer: A constructor is a special block of code used to initialize objects.

- It has no return type (not even void).
- The name must match the class name.

```
class Car {
    String brand;
    int speed;
    Car(String brand, int speed) { // constructor
        this.brand = brand;
        this.speed = speed;
    }
}
Car myCar = new Car("BMW", 120);
```

Tip: Always mention constructors are implicitly called when new is used.

12. What is the difference between a constructor and a method?

Constructor	Method
No return type	Has return type (even void)
Name = Class name	Name = any valid identifier
Called automatically when object is created	Called explicitly
Used to initialize object	Used to perform operations

13. What are the types of constructors in Java?

1. Default Constructor (No-Arg Constructor): Takes no arguments and initializes objects with default values.

```
class Car {  
    String color;  
    Car() { color = "White"; }  
}
```

2. Parameterized Constructor: Accepts arguments to initialize an object with specific values.

```
class Car {  
    String color;  
    Car(String c) { color = c; }  
}
```

3. Copy Constructor (User-defined): Creates a new object by copying values from another object.

```
class Car {  
    String color;  
    Car(Car c) { color = c.color; }  
}
```

14. What is a default constructor? When is it created by the compiler?

Answer: A default constructor is a no-argument constructor automatically provided by the compiler if no constructors are defined in the class. If you define any constructor, the compiler does not create a default one.

```
class Car { } // compiler adds implicitly: Car() {}
```

15. Can a constructor be private? If yes, where is it useful?

Answer: Yes. Private constructors are used for Singleton design pattern or static utility classes (like Math).

```
class Singleton {  
    private static Singleton instance = new Singleton();  
    private Singleton() {} // private constructor  
    public static Singleton getInstance() { return instance; }  
}
```

16. Can a class have multiple constructors? How?

Answer: Yes, constructor overloading allows multiple constructors with different parameters.

```
class Car {  
    String brand;  
    int speed;  
    Car() { brand = "Unknown"; speed = 0; }  
    Car(String brand) { this.brand = brand; speed = 0; }  
    Car(String brand, int speed) { this.brand = brand; this.speed = speed; }  
}
```

17. Can constructors be overloaded?

Answer: Yes, just like methods, constructors can have different parameter lists to allow creating objects in different ways.

18. What happens if you define a parameterized constructor but not a default one?

Answer: The compiler does not provide a default constructor. Trying to create an object with no arguments will cause a compile-time error.

```
class Car { Car(String brand) {} }  
// Car c = new Car(); // Error!
```

19. Can a constructor call another constructor? How?

Answer: Yes, use `this()` to call another constructor in the same class. It must be the first statement in the constructor.

```
class Car {  
    String brand;  
    int speed;  
    Car() { this("Unknown", 0); } // calls parameterized constructor  
    Car(String brand, int speed) {  
        this.brand = brand;  
        this.speed = speed;  
    }  
}
```

20. What is constructor chaining?

Answer: Constructor chaining is calling one constructor from another (either within the same class using `this()` or from the parent class using `super()`). Helps avoid code duplication.

21. Can we inherit constructors from the parent class?

Answer: No, constructors are not inherited. A subclass can call a parent constructor using `super()`, but cannot directly inherit it.

C. Static, this Keyword, Instance vs Class Variables, Getters & Setters

22. What is this keyword in Java?

Answer: `this` refers to the current object. Used to differentiate instance variables from parameters, call other constructors, or pass current object instance.

```
class Person {  
    String name;  
    Person(String name) { this.name = name; }  
    void printPerson() { System.out.println(this.name); }  
}
```

23. What is a static variable in Java?

Answer: Belongs to the class, not individual objects. Shared across all instances and stored in the method area (class memory).

24. What is a static method in Java?

Answer: It belongs to the class, can be called without creating an object, & can access only static variables/methods. Cannot use this.

25. Difference between instance variables and static variables?

Instance Variable	Static Variable
Belongs to object	Belongs to class
Unique per object	Shared across all objects
Stored in heap	Stored in method area
Accessed via object reference	Accessed via class name

26. What are getters and setters? Why do we use them?

Answer: Methods used to access and modify private variables (encapsulation). Useful for validation and controlling access.

27. Can a static method access instance variables?

Answer: No, because instance variables belong to objects, while static methods belong to the class. To access instance variables, pass an object reference explicitly.

28. Can this be used in a static method?

Answer: No, this refers to current object, but static methods don't belong to any specific object instance.

29. What happens if you don't use getters/setters and make variables public?

Answer: Anyone can modify data directly without validation -> Encapsulation is broken -> Higher chance of bugs & invalid states.

30. Can static variables and methods be accessed via objects?

Answer: Yes, but it is not recommended. Accessing via the class name (e.g. Demo.count) is preferred to indicate shared nature.

STAGE 2: Encapsulation

A. Core Concept

31. What is encapsulation in Java?

Answer: Encapsulation is wrapping data (variables) and behavior (methods) together into a single unit and restricting direct access to internal state.

```
class Person {  
    private String name; // hidden  
    public void setName(String name) { this.name = name; }  
    public String getName() { return name; }  
}
```

32. Why is encapsulation important in OOP?

- Protects object state from unintended/invalid modifications.
- Makes code maintainable, flexible, and modular.

33. How does encapsulation differ from abstraction?

- **Encapsulation:** Hides implementation details / data at object level using access modifiers.
- **Abstraction:** Hides complexity at class/method level, showing only what an object does.

34. What is data hiding and how does Java achieve it?

Answer: Data hiding restricts external classes from directly accessing state. Achieved via private access modifiers + public getters/setters.

35. What are the key features that enable encapsulation in Java?

- Private variables
- Public/protected getters and setters
- Final keyword for immutable fields
- Access modifiers (public, private, protected, default)

36. Can encapsulation exist without access modifiers?

Answer: Not fully. While package-private default visibility exists, true encapsulation requires fine-grained control provided by private/protected.

37. Is encapsulation only about making variables private?

Answer: No. Encapsulation also involves providing controlled access through methods, validating inputs, and enforcing field immutability where appropriate.

38. How does encapsulation improve code maintainability and flexibility?

Answer: Internal implementation can change without affecting calling code (e.g., updating salary calculation logic inside `getSalary()`).

39. What will happen if we make instance variables public?

Answer: Direct modification can cause invalid states, break object integrity, and tightly couple classes.

40. What are the disadvantages of not using encapsulation?

Answer: Increased risk of invalid data, difficult code maintenance, poor modularity, and high bug rate in team projects.

B. Practical Implementation

41. What are getters and setters in Java?

Getters: Methods that return field values. **Setters:** Methods that modify field values with validation.

42. Why do we use getters and setters instead of accessing variables directly?

Answer: To validate input, hide internal data structures, and allow future internal changes without breaking client code.

43. Can we have only a getter or only a setter in a class? Why?

Answer: Yes. Getter-only creates read-only properties (e.g. ID). Setter-only creates write-only properties (e.g. password).

44. Can you write a simple Java class that demonstrates encapsulation?

```
class Employee {
    private String name;
    private double salary;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public double getSalary() { return salary; }
    public void setSalary(double salary) {
        if(salary > 0) this.salary = salary;
        else System.out.println("Invalid salary");
    }
}
```

45. How do access modifiers affect encapsulation?

- **Private:** Fully hidden (best for encapsulation)
- **Protected:** Accessible to subclasses
- **Default:** Accessible within package
- **Public:** Accessible everywhere (breaks encapsulation on variables)

46. What is a POJO and how does it represent encapsulation?

Answer: Plain Old Java Object. Uses private fields and public getters/setters to hide internal data.

47. How does encapsulation support immutability? Give an example.

```
final class Person {  
    private final String name;  
    Person(String name) { this.name = name; }  
    public String getName() { return name; } // No setters -> Immutable  
}
```

48. How can you make a class immutable using encapsulation?

- Make class `final`.
- Make fields `private final`.
- Provide no setters, only getters.
- Deep clone mutable objects in getters/constructors.

49. How does encapsulation help in teams or large codebases?

Answer: Reduces side-effects, isolates bug locations, and allows parallel development on decoupled modules.

50. Give a real-world example representing encapsulation.

Answer: An ATM machine hides internal cash handling & software logic. Users interact exclusively via predefined screens & buttons (public interface).

STAGE 3: Inheritance

A. Core Concepts

51. What is inheritance in Java?

Answer: Mechanism where a subclass acquires properties/behaviors of a superclass using `extends`.

```
class Animal { void eat() { System.out.println("Animal eats"); } }  
class Dog extends Animal { void bark() { System.out.println("Dog barks"); } }
```

52. Why do we use inheritance?

- Code Reuse & Extensibility
- Enables Polymorphism
- Establishes logical hierarchy

53. What is the difference between IS-A and HAS-A relationships?

Type	Description	Example
IS-A	Inheritance relationship (subclass is a superclass)	Dog IS-A Animal
HAS-A	Composition/Aggregation (class holds instance of another)	Car HAS-A Engine

54. What different types of inheritance exist in Java? Explain each.

1. **Single Inheritance:** Class A -> Class B
2. **Multilevel Inheritance:** Class A -> Class B -> Class C
3. **Hierarchical Inheritance:** Class A -> Class B & Class C
4. **Multiple Inheritance (Interfaces only):** Interface A & B -> Class C

55. Why is multiple inheritance not allowed in Java using classes?

Answer: To avoid ambiguity and the *Diamond Problem* (conflicting duplicate methods from multiple parent classes).

56. How does Java support multiple inheritance using interfaces?

Answer: A class can implement multiple interfaces because interfaces specify contracts without implementation conflicts.

57. What is the super keyword? Give an example.

Answer: Reference to the superclass object. Used to access parent methods, parent fields, or invoke parent constructor.

```
class Parent { int x = 10; }
class Child extends Parent {
    int x = 20;
    void print() { System.out.println(super.x); } // prints 10
}
```

58. Can private members of a superclass be inherited? Why or why not?

Answer: Private members are not directly accessible/visible in subclasses, but can be accessed indirectly via public/protected getters.

59. Can constructors be inherited?

Answer: No, constructors are specific to a class and cannot be inherited, though they can be called via `super()`.

60. What is constructor chaining in inheritance?

Answer: Calling superclass constructors sequentially down the inheritance tree before executing subclass constructor logic.

B. Implementation / Coding Questions - Inheritance

61. Write a simple example of single inheritance (Parent -> Child) in Java.

```
class Parent { void greet() { System.out.println("Hello from Parent"); } }
class Child extends Parent { void greetChild() { System.out.println("Hello from Child"); } }
```

62. Write an example of multilevel inheritance (GrandParent -> Parent -> Child).

```
class GrandParent { void msg1() { System.out.println("GrandParent"); } }
class Parent extends GrandParent { void msg2() { System.out.println("Parent"); } }
class Child extends Parent { void msg3() { System.out.println("Child"); } }
```

63. Show how to call the superclass constructor from a subclass.

```
class Parent { Parent() { System.out.println("Parent Constructor"); } }
class Child extends Parent {
    Child() {
        super(); // Call parent constructor
        System.out.println("Child Constructor");
    }
}
```

64. Can we override a static method in Java? Explain with an example.

Answer: No. Static methods belong to class level and are resolved at compile time. Redefining them in a subclass is *method hiding*, not overriding.

65. What happens if a subclass defines a method with the same name but different parameters?

Answer: This is method overloading across inherited levels, not method overriding.

66. Can a subclass access protected members of the superclass in another package?

Answer: Yes, protected members are accessible across packages through inheritance (via subclass reference).

67. Write an example demonstrating method overriding in Java.

```
class Parent { void show() { System.out.println("Parent show"); } }  
class Child extends Parent {  
    @Override  
    void show() { System.out.println("Child show"); }  
}
```

68. Explain what happens when a subclass hides a superclass field with the same name.

Answer: Field hiding occurs. Field access is resolved based on reference variable type, not the underlying object type.

69. How do you prevent a class from being subclassed?

Answer: Declare the class using the `final` keyword (e.g., `final class Parent {}`).

70. Give a real-world example where inheritance improves code reuse and maintainability.

Answer: A base `Vehicle` class defining `start()`/`stop()` inherited by `Car`, `Bike`, and `Truck` avoiding redundant duplicate methods.

STAGE 4: Polymorphism

71. What is polymorphism in Java?

Answer: Ability of an object or method to take on multiple forms, enabling dynamic behavior execution.

72. Why is polymorphism important in OOP?

Answer: Decouples interface design from concrete implementations, enabling flexible and maintainable systems.

73. What are the two types of polymorphism in Java?

- **Compile-time (Static):** Method Overloading
- **Runtime (Dynamic):** Method Overriding

74. What is compile-time (static) polymorphism?

Answer: Method call resolved at compile time based on parameter signatures (Method Overloading).

75. What is runtime (dynamic) polymorphism?

Answer: Method call resolved at runtime by JVM based on actual object type (Method Overriding).

76. How does method overloading achieve compile-time polymorphism?

Answer: Same method name with different parameters resolved during compilation.

77. How does method overriding achieve runtime polymorphism?

Answer: Subclass provides custom implementation executed at runtime via superclass reference variable.

78. Can constructors be overloaded? Is it polymorphism?

Answer: Yes, constructors can be overloaded. It is compile-time polymorphism.

79. Can private methods be overridden? Why or why not?

Answer: No, private methods are not visible outside their class scope.

80. What is a covariant return type in method overriding?

Answer: Allows an overridden method in a subclass to return a subtype of the original superclass method's return type.

81. What is the difference between method overloading and method overriding?

Feature	Method Overloading	Method Overriding
Polymorphism	Compile-time (Static)	Runtime (Dynamic)
Parameters	Must be different	Must be identical
Class Scope	Same class (or subclass)	Subclass extending superclass
Return Type	Can differ	Must be same or covariant

82. Can private, final, or static methods be overridden? Why or why not?

Answer: None can be overridden. Private = not visible; Final = prevents modification; Static = bound to class, not instance.

83. Difference between compile-time and runtime polymorphism in terms of method binding?

Compile-time uses early static binding by compiler; Runtime uses late dynamic binding by JVM.

84. How does Java resolve which method to call at runtime in case of overriding?

Uses Dynamic Method Dispatch based on the underlying object type in heap memory.

85. Can constructors participate in runtime polymorphism? Why or why not?

No, constructors cannot be overridden and are called explicitly during instantiation.

86. What is dynamic method dispatch in Java, and how is it related to polymorphism?

Mechanism where overridden method call is resolved dynamically at runtime.

87. How does polymorphism improve code maintainability and flexibility in large applications?

Allows adding new implementations without modifying existing business logic code relying on abstractions.

88. What is the difference between polymorphism in Java and C++?

Feature	Java	C++
Virtual methods	All non-final instance methods are virtual by default	Requires explicit virtual keyword
Multiple inheritance	Interfaces only	Supported directly for classes

89. Can you override a superclass method and change its access modifier? What are the rules?

You can increase visibility (e.g. protected -> public), but you cannot reduce visibility.

90. Give a real-world analogy to explain polymorphism in Java.

A Universal Remote Control: Power button sends same command, but TV, AC, and Soundbar respond according to their custom behavior.

STAGE 5: Abstraction

91. What is abstraction in Java?

Hiding complex internal implementation details and showing only essential user functionality.

92. How is abstraction different from encapsulation?

Abstraction	Encapsulation
Hides operational implementation complexity	Hides internal state data directly
Achieved via Abstract classes & Interfaces	Achieved via private variables & getters/setters

93. Why do we need abstraction in OOP?

Reduces code complexity, enforces architectural contracts, and promotes modular reusability.

94. What is an abstract class in Java?

A class declared with `abstract` keyword that cannot be directly instantiated and can contain abstract/concrete methods.

95. Can an abstract class have a constructor? If yes, why is it useful?

Yes, to initialize common parent fields shared across subclasses during subclass instantiation.

96. Can an abstract class have both abstract and non-abstract (concrete) methods?

Yes, it supports partial abstraction.

97. Can you instantiate an abstract class? Why or why not?

No, because it may contain incomplete method implementations.

98. What is an interface in Java?

A pure abstraction blueprint defining standard method contracts class implementations must satisfy.

99. Can interfaces have method implementations in Java?

Yes, starting Java 8+ using default and static methods, and Java 9+ using private helper methods.

100. What is the difference between an abstract class and an interface in Java?

Feature	Abstract Class	Interface
Multiple Inheritance	No (Single class extension)	Yes (Implement multiple)
Variables	Instance variables allowed	Only <code>public static final</code> constants
Constructors	Allowed	Not allowed

101. Can a class extend multiple abstract classes?

No, Java does not support multiple class inheritance.

102. Can a class implement multiple interfaces?

Yes, enabling multiple type inheritance without implementation diamond issues.

103. Can an interface extend another interface? Can it extend multiple interfaces?

Yes, an interface can extend multiple interfaces using `extends`.

104. Can an abstract class implement an interface?

Yes, it can implement interface methods or leave them abstract for child classes.

105. What is the purpose of default methods in interfaces?

To enable backward compatibility by adding new methods to interfaces without breaking existing implementing classes.

106. Can static methods in interfaces be overridden in implementing classes?

No, static interface methods belong to interface type directly.

107. Can a private method exist in an interface? How is it used?

Yes (Java 9+), used to encapsulate helper code shared between default methods.

108. Can you declare variables in an interface? What are their default modifiers?

Yes, implicitly `public static final` constants.

109. How does abstraction help achieve polymorphism in Java?

Provides common abstract type references that dynamically point to various concrete subclass instances.

110. How does abstraction improve maintainability, flexibility, and scalability?

Decouples specifications from concrete implementations, allowing isolated upgrades.

STAGE 6: Object Relationships

111. What is object association in Java?

Relationship representing "uses-a" or "works-with" connectivity between independent objects.

112. What is aggregation? How is it different from association?

Weak "HAS-A" relationship with shared ownership where both parent and child can exist independently (e.g. Library & Books).

113. What is composition? How is it different from aggregation?

Strong "HAS-A" relationship with strict lifetime dependency (e.g. House & Rooms).

114. Can you give real-world examples of association, aggregation, and composition?

Type	Example	Lifetime Dependency
Association	Teacher & Student	Independent
Aggregation	Library & Book	Independent
Composition	House & Room	Dependent

115. What is the difference between HAS-A and IS-A relationships?

IS-A: Inheritance (via extends). **HAS-A:** Association/Composition (via instance variables).

116. When should you prefer composition over inheritance?

Prefer composition for flexibility, loose coupling, and dynamic component swapping at runtime.

117. What is dependency in OOP?

Temporary association where one class briefly uses another inside a method scope.

118. How does dependency differ from aggregation?

Dependency is short-lived method level; Aggregation is long-lived structural class member level.

119. Can a class have multiple associations with another class?

Yes, representing different roles (e.g. Teacher teaches Student, Teacher mentors Student).

120. Can aggregation exist without a "has-a" relationship?

No, aggregation inherently implies a shared whole-part ownership HAS-A structure.

STAGE 7: Important OOP Keywords

121. Can a constructor be declared final, static, or abstract? Why or why not?

No. Final makes no sense (not inherited), static invalid (constructor builds instances), abstract invalid (must have body).

122. Can we use this() and super() together in the same constructor?

No, both must be the first statement inside a constructor body.

123. Difference between static block and instance initializer block?

Static Block	Instance Initializer Block
Executes once when class loads	Executes every time an object is instantiated
Cannot access instance variables	Can access instance variables

124. What happens if you declare a static variable inside a method?

Compile-time error. Local variables live on stack; static variables belong to class memory area.

125. Can a class be both abstract and final at the same time?

No, contradictory semantics (abstract requires subclasses, final prevents subclassing).

126. Can a static method be abstract? Why or why not?

No, static methods cannot be overridden.

127. Can a static method access non-static data members or methods?

Not directly. It must instantiate an object first.

128. What happens if you call this() or super() from a normal method?

Compile error. Constructor invocation statements are restricted strictly to constructor bodies.

129. Can you override a method and make it static in the subclass?

No, static methods participate in method hiding, not overriding.

130. What happens if you make an interface variable non-final or non-static?

Not allowed; interface variables are implicitly `public static final`.

STAGE 8: Comparison-Based Topics

131. Difference between Method Overloading and Method Overriding

Aspect	Overloading	Overriding
Binding Time	Compile-time	Runtime
Parameters	Must differ	Must be identical
Inheritance	Not required	Required

132. Difference between Compile-time Polymorphism and Runtime Polymorphism

Feature	Compile-time	Runtime
Achieved By	Method Overloading	Method Overriding
Binding	Static / Early Binding	Dynamic / Late Binding
Resolved By	Compiler	JVM
Speed	Faster	Slightly Slower

133. Difference between Static Binding and Dynamic Binding

Static binding happens at compile time for static, final, or private methods. Dynamic binding happens at runtime for overridden instance methods.

134. Difference between Constructor Overloading and Method Overloading

Constructor overloading initializes objects using different parameter sets; Method overloading executes different method operations using different parameter sets.

135. Difference between Constructor and Method Overriding

Constructors cannot be overridden because they are not inherited by subclasses. Methods can be overridden.

136. Difference between Instance Method and Static Method in Polymorphism

Instance methods support dynamic polymorphism; Static methods use static binding (method hiding) without runtime polymorphism.

137. Difference between Superclass Reference and Subclass Reference Behavior

Superclass reference allows accessing superclass methods and invoking overridden subclass implementations dynamically. Subclass reference accesses both superclass and subclass specific methods directly.

138. Difference between Abstract Class and Interface

Aspect	Abstract Class	Interface
Abstraction	Partial (0-100%)	Full contract
Multiple Inheritance	Not supported	Supported
Constructors	Allowed	Not allowed

139. Difference between Final Class and Abstract Class

Final class cannot be extended. Abstract class must be extended.

140. Difference between Interface Inheritance and Class Inheritance

Interface inheritance allows multiple behavior contracts without concrete state; Class inheritance provides single hierarchy extension with state and concrete code.

Resource & Reference Links

- Instagram: [instagram.com/codewithnishchal](https://www.instagram.com/codewithnishchal)
- LinkedIn: [linkedin.com/in/nishchal-muradia](https://www.linkedin.com/in/nishchal-muradia)
- YouTube: [youtube.com/@CodeWithNishchal](https://www.youtube.com/@CodeWithNishchal)